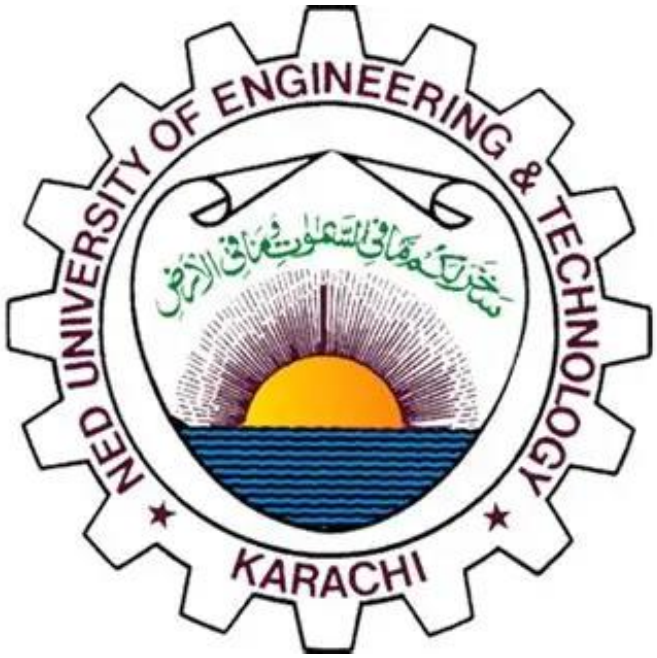



**NED UNIVERSITY OF ENGINEERING AND TECHNOLOGY**

Department of Computer Science & Information Technology

*Bachelor of Science in Computer Science (Specialization in Cyber Security)*

---

**COMPLEX COMPUTING ACTIVITY (CCA) REPORT**

# PhishGuard

## Multi-Source Email Threat Classification

*Comprehensive Technical Report & Engineering Architecture*

### Course Details

| | |
|---|---|
| **Course Code and Title** | CT-264 – Programming for Artificial Intelligence |
| **Assessment Type** | Complex Computing Activity (CCA) through Problem-Based Learning (PBL) |
| **Course Learning Outcome** | CLO 3 (C3) – Identify appropriate AI techniques to solve classification problems of moderate complexity |
| **Project Title** | PhishGuard: Multi-Source Email Threat Classification |

### Group Members



| Roll Number | Student Name |
|---|---|
| **CT-24278** | Muhammad Saad |
| **CT-24125** | Haris Zamir |
| **CT-24136** | Ali Hassan |
| **CT-24138** | Muhammad Raza |

# Table of Contents

**1. Abstract** ..... **3**

**2. Introduction & Problem Statement** ..... **3**
2.1 The Evolving Phishing Threat Landscape ..... 3
2.2 Shortcomings of Single-Source Corpora ..... 3
2.3 Objectives of PhishGuard ..... 3

**3. Dataset Analysis & Preprocessing** ..... **4**
3.1 Multi-Source Corpus Aggregation ..... 4
3.2 Dynamic Column Inspection & Mapping Pipeline ..... 4
3.3 Data Hygiene & Deduplication ..... 5

**4. Exploratory Data Analysis (EDA)** ..... **5**
4.1 Class Distribution ..... 5
4.2 Text Length Profiling ..... 6
4.3 Discriminative Lexical Analysis ..... 7

**5. Methodology** ..... **7**
5.1 Data Partitioning ..... 8
5.2 TF-IDF Feature Representation ..... 8
5.3 Model Selection: Regularized Logistic Regression ..... 9

**6. Results & Evaluation** ..... **9**
6.1 Test Set Classification Performance ..... 9
6.2 Confusion Matrix Analysis ..... 9
6.3 Qualitative Stress Testing & Concept Drift Analysis ..... 10

**7. Challenges & Engineering Solutions** ..... **10**

**8. Interactive Deployment (Streamlit Application)** ..... **11**
8.1 Interface Features ..... 11
8.2 Local Launch Instructions ..... 11

**9. Conclusion & Future Roadmap** ..... **11**

**10. References** ..... **12**

# 1. Abstract

Phishing remains one of the primary attack vectors in modern cybersecurity breaches, exploiting social engineering techniques to deceive users into disclosing sensitive personal, financial, or corporate credentials. While machine learning and natural language processing (NLP) have shown immense promise in detecting email-based threats, real-world deployments are routinely hindered by isolated, domain-specific training corpora that suffer from class imbalance, contextual fragility, and schema incompatibility.

In this work, we propose and implement **PhishGuard**, an end-to-end multi-source email threat classification system engineered to aggregate, standardize, clean, and classify heterogeneous email corpora. By integrating seven prominent benchmark datasets (CEAS 2008, Enron, Ling-Spam, Nazario Phishing Corpus, Nigerian Fraud, Kaggle Phishing Email, and SpamAssassin), PhishGuard constructs a consolidated corpus of **163,868 verified emails** with a balanced distribution (51.90% threat vs. 48.10% benign). Using sublinear Term Frequency–Inverse Document Frequency (TF-IDF) features spanning 10,000 unigrams and bigrams, paired with an $L_2$-regularized Logistic Regression classifier, the system achieves **98.60% test accuracy**, **98.34% precision**, **98.97% recall**, and an **F1-score of 0.9865**.

Furthermore, an interactive web interface built with Streamlit provides real-time, low-latency inference and confidence estimation. This report provides a detailed analysis of the data ingestion pipeline, exploratory data findings, feature engineering methodology, empirical evaluation, edge-case stress testing, technical challenges encountered, and architectural considerations for production readiness.

# 2. Introduction & Problem Statement

## 2.1 The Evolving Phishing Threat Landscape

Social engineering via electronic mail accounts for over 80% of reported security incidents globally. Adversaries routinely deploy spoofed sender identities, homograph domain attacks, urgency-inducing psychological triggers, and disguised hyperlinks leading to credential-harvesting pages or malware delivery endpoints. Traditional signature-based defenses (such as hash matching and rigid heuristic keyword rule sets) frequently fail against zero-day campaigns, paraphrased attacks, and obfuscated text.

## 2.2 Shortcomings of Single-Source Corpora

Existing automated phishing detection studies frequently rely on a single isolated dataset:

1. **The Enron Dataset:** Reflects corporate communications from an energy corporation in the early 2000s; contains authentic benign business language but lacks contemporary attack variants.
2. **The Nazario Corpus:** Contains genuine, highly sophisticated targeted phishing attacks, but suffers from a small sample size (fewer than 2,000 samples).
3. **SpamAssassin / Ling-Spam:** Focus primarily on generic unsolicited commercial spam or academic mailing lists rather than targeted spear-phishing or financial wire fraud.

Training an AI classifier on any single source leads to severe domain overfitting. A model trained on Enron may over-index on energy terminology, while a model trained solely on Nazario fails to distinguish routine administrative notices from phishing attempts.

### 2.3 Objectives of PhishGuard

The core objective of the PhishGuard project is to construct a scalable, production-oriented text classification pipeline that:

- Ingests multiple heterogeneous, real-world data sources without manual intervention.
- Resolves conflicting schemas, encoding variations, and inconsistent target labeling dynamically.
- Eliminates duplicate entries and corrupt texts to establish a clean dataset.
- Analyzes statistical distribution differences between threat vectors and legitimate traffic.
- Trains a resilient, high-throughput machine learning baseline capable of high-precision classification.
- Delivers an intuitive graphical interface for operational deployment and edge-case stress testing.

## 3. Dataset Analysis & Preprocessing

### 3.1 Multi-Source Corpus Aggregation

To achieve broad lexical diversity and historical coverage, PhishGuard aggregates seven distinct datasets placed in `data/raw/` (Table 3.1).

**Table 3.1:** *Source datasets used to build the PhishGuard corpus.*

| Dataset Source | Original Rows | Raw Schema Columns | Primary Focus |
|---|---|---|---|
| **`CEAS_08.csv`** | 39,154 | `sender, receiver, date, subject, body, label, urls` | Anti-Spam Challenge (spam + phishing) |
| **`Enron.csv`** | 29,767 | `subject, body, label` | Corporate communications (benign baseline) |
| **`Ling.csv`** | 2,859 | `subject, body, label` | Linguistic mailing list (benign baseline) |
| **`Nazario.csv`** | 1,565 | `sender, receiver, date, subject, body, urls, label` | Targeted, confirmed phishing emails |
| **`Nigerian_Fraud.csv`** | 3,332 | `sender, receiver, date, subject, body, urls, label` | Advance-fee fraud (419 scams) |
| **`phishing_email.csv`** | 82,486 | `text_combined, label` | Aggregated Kaggle benchmark collection |
| **`SpamAssasin.csv`** | 5,809 | `sender, receiver, date, subject, body, label, urls` | Classic open-source filtering corpus |
| **Total concatenated** | **164,972** | *Heterogeneous* | *Comprehensive multi-vector corpus* |

### 3.2 Dynamic Column Inspection & Mapping Pipeline

Each raw file exhibited differing column conventions. For instance:

- Email body text appeared variously under `body`, `text_combined`, `text`, or `message`.
- Target threat labels were stored as integers (`0/1`), floats (`0.0/1.0`), or strings (`ham/spam`).
- Ancillary metadata (`sender`, `receiver`, `date`, `urls`) frequently contained substrings like `email` or `mail` that could trigger false-positive column mapping if naive substring matching was employed.

To solve this, `src/data_loader.py` implements a priority-driven heuristic mapping algorithm:

1. **Exact candidate evaluation:** Checks for standardized candidates in prioritized order (`['text_combined', 'body', 'text', 'message', 'content']` for text; `['label', 'class', 'target', 'spam']` for labels).
2. **Context-aware substring matching:** If exact matching fails, scans column names against approved keywords while strictly excluding non-body fields (`sender`, `receiver`, `to`, `from`, `date`, `url`, `id`, `subject`).
3. **Encoding auto-recovery:** Implements fallback decoding from `utf-8` to `latin1` and `utf-8-sig` with replacement strategies to prevent pipeline crashes on corrupted SMTP byte streams.

### 3.3 Data Hygiene & Deduplication

Following column alignment:

1. **Null/missing record removal:** Records with null values or empty whitespace-only text strings were purged, eliminating 6 corrupted entries (164,966 remaining).
2. **Exact duplicate purging:** Multi-source aggregation inevitably produces duplicate cross-posted messages and template variations. Applying deduplication across the normalized text column removed **1,098 duplicate emails**.
3. **Label canonicalization:** Non-standard string representations (`ham`, `clean`, `benign` → 0; `phishing`, `spam`, `fraud` → 1) were cast to binary integers.

The final cleaned master dataset was saved to `data/processed/cleaned_phishing_data.csv` with **163,868 rows** and **2 standardized columns** (`text`, `label`).

## 4. Exploratory Data Analysis (EDA)

Exploratory analysis was conducted in `notebooks/01_EDA_and_Cleaning.ipynb` to investigate class distribution, text length distributions, and discriminative lexical features.

### 4.1 Class Distribution

A crucial finding of the multi-source alignment was the natural balance achieved across classes (Table 4.1, Figure 4.1).

**Table 4.1:** *Class distribution of the consolidated corpus.*

| Class Label | Designation | Email Count | Percentage |
|---|---|---|---|
| **0** | Legitimate / Benign | 78,815 | **48.10%** |
| **1** | Phishing / Threat | 85,053 | **51.90%** |
| **Total** | | **163,868** | **100.0%** |

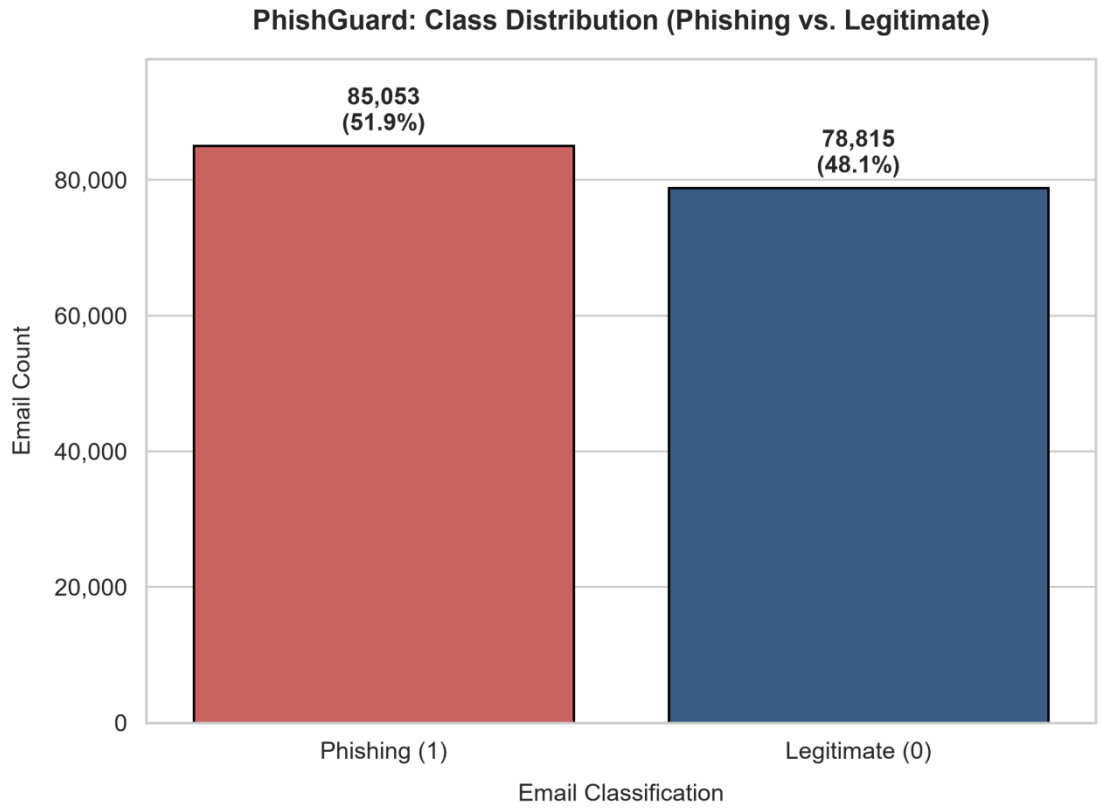



**Figure 4.1:** *Class distribution of the consolidated PhishGuard dataset.*

Because the dataset naturally aligns at an approximately 52/48 balance, artificial oversampling techniques (such as SMOTE or ADASYN) were avoided. This preserves authentic language variance and prevents synthetic artifacts.

## 4.2 Text Length Profiling

Text length was computed both in character count (`email_length`) and whitespace-separated tokens (`word_count`), as summarized in Table 4.2.

**Table 4.2:** *Descriptive statistics of email length by class.*

| Statistical Metric | Legitimate (Class 0) | Phishing (Class 1) | Combined Corpus |
|---|---|---|---|
| **Count** | 78,815 | 85,053 | 163,868 |
| **Median Characters** | **908.0** | **428.0** | **614.0** |
| **Mean Characters** | 1,851.5 | 1,232.3 | 1,530.1 |
| **Standard Deviation** | 4,639.2 | 22,160.5 | 16,289.1 |
| **Median Word Count** | 143.0 | 66.0 | 97.0 |
| **Mean Word Count** | 267.3 | 171.6 | 217.7 |

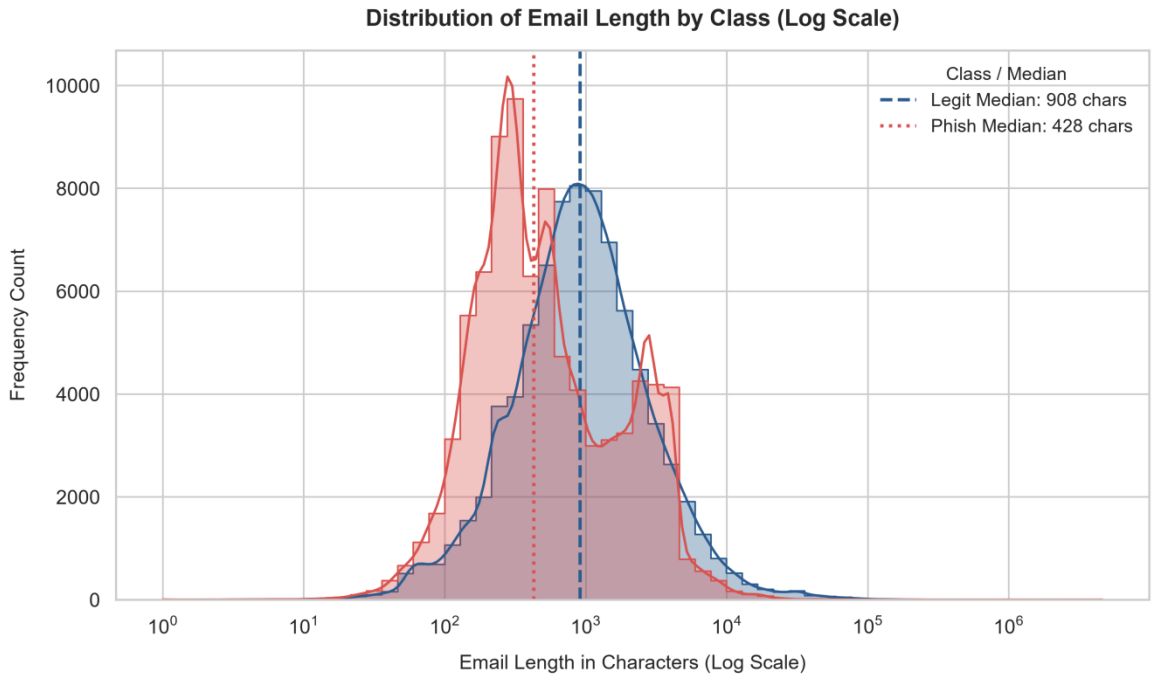



**Figure 4.2:** *Log-scale distribution of email lengths (characters) for threat and benign classes.*

**Key observations:**

1. **Urgency and conciseness:** Phishing emails exhibit a median length of **428 characters**—less than half that of legitimate corporate and personal correspondence (**908 characters**). Threat actors construct concise, directive prompts designed to induce a rapid response before critical scrutiny occurs.
2. **Extreme positive skewness:** Both classes feature extreme outliers (up to 127,119 words) stemming from automated log exports and MIME base64 dump files. Plotting on a **logarithmic scale** (Figure 4.2) reveals distinct peaks that separate threat vectors from operational communications.

### 4.3 Discriminative Lexical Analysis

To identify dominant vocabulary features, term-frequency analysis was performed across both partitions after filtering out standard English stop words (Figure 4.3).

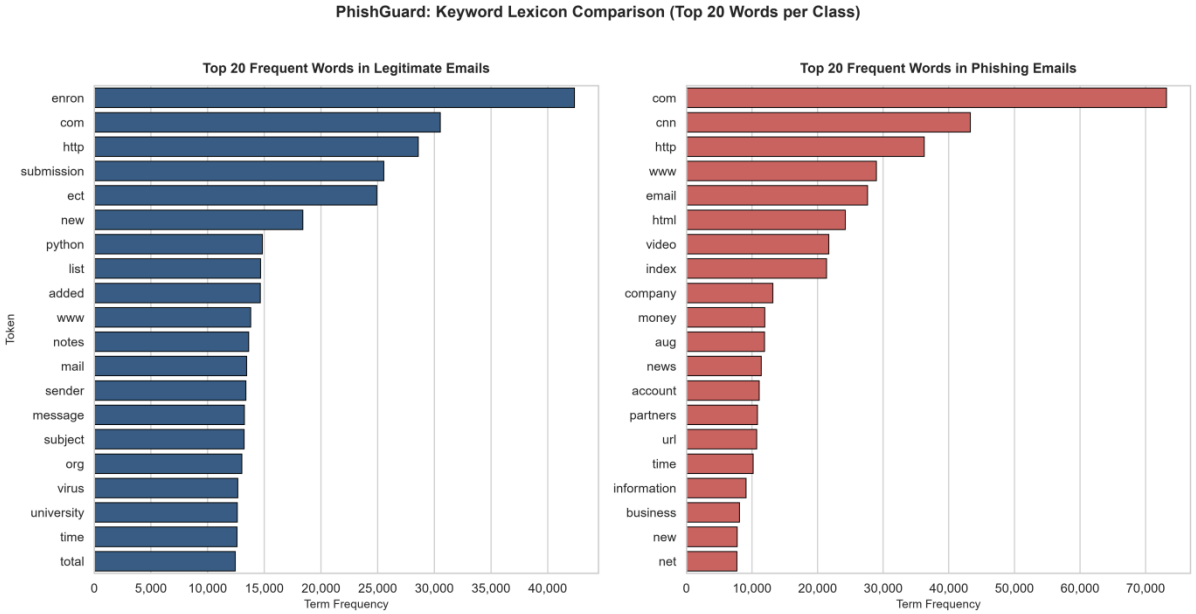



**Figure 4.3:** *Top 20 most frequent words in legitimate vs. phishing emails.*

- **Phishing lexicon:** Dominated by web and link-related tokens (`com`, `http`, `www`, `html`, `url`, `index`) together with account- and finance-oriented terms (`email`, `account`, `money`, `information`, `company`, `business`).
- **Legitimate lexicon:** Centered on mailing-list and workplace communication artifacts (`enron`, `submission`, `ect`, `list`, `notes`, `mail`, `sender`, `message`, `subject`) and technical or academic discussion (`python`, `virus`, `university`).

This lexical divergence confirms that term-frequency representations possess strong discriminative capability for downstream classification models. It also shows that some highly frequent tokens (such as `enron` and `cnn`) are tied to individual source corpora rather than to phishing behavior itself; this source-specific signal is a limitation discussed in Section 6.3.

## 5. Methodology

The machine learning pipeline in `src/preprocess.py` and `src/train.py` was designed for generalization, computational efficiency, and maintainability. Figure 5.1 summarizes the end-to-end flow.

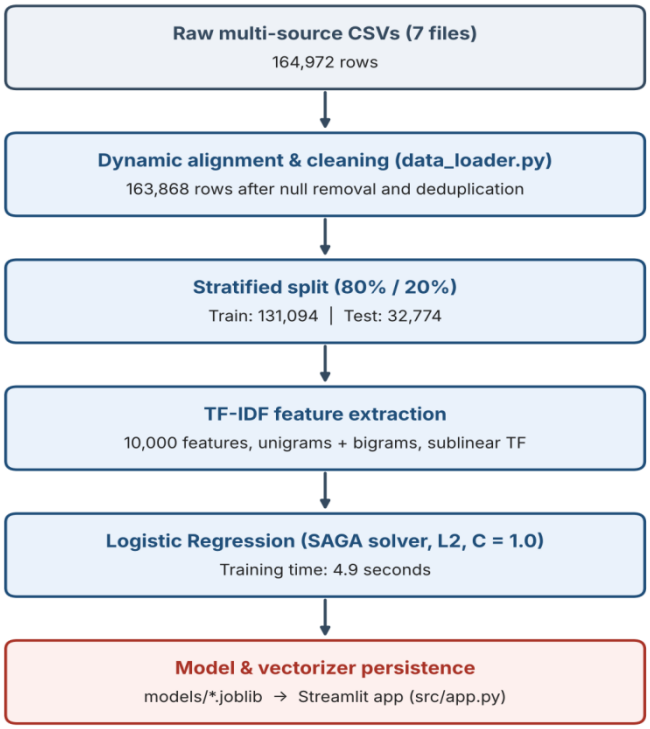



**Figure 5.1:** *PhishGuard data and model pipeline.*

## 5.1 Data Partitioning

The dataset was split using stratified sampling with a fixed random seed (`random_state=42`):

- **Training set (80%):** 131,094 emails (68,042 phishing, 63,052 legitimate)
- **Test set (20%):** 32,774 emails (17,011 phishing, 15,763 legitimate)

Stratification ensures that the test evaluation accurately mirrors the 51.90% / 48.10% ground-truth class ratio.

## 5.2 TF-IDF Feature Representation

Emails were vectorized using scikit-learn's `TfidfVectorizer`, configured as follows:

- **`max_features=10,000`:** Caps the vocabulary at the 10,000 most informative tokens, preventing memory exhaustion while retaining granular lexical signal.
- **`ngram_range=(1, 2)`:** Captures both isolated words and two-word collocations (e.g., `verify account`, `click here`, `security alert`, `wire transfer`).
- **`sublinear_tf=True`:** Replaces term frequency *tf* with 1 + log(*tf*) for positive occurrences, dampening the influence of highly repetitive terms in long documents.
- **`min_df=3` / `max_df=0.95`:** Eliminates ultra-rare typographical errors and corpus-wide invariant terms.
- **`token_pattern=r'(?u)\b[a-zA-Z]{3,}\b'`:** Removes single- and double-letter noise while capturing semantic tokens.
- **Strict leakage prevention:** The vectorizer was fitted solely on the training partition ($X_{\text{train}}$) and used in transform-only mode on the held-out test partition ($X_{\text{test}}$).

### 5.3 Model Selection: Regularized Logistic Regression

A linear classification architecture was selected based on several engineering criteria:

1. **Sparsity suitability:** Sparse TF-IDF text matrices with 10,000 dimensions are typically well separated by linear hyperplanes in high-dimensional space.
2. **Inference latency:** Evaluating an incoming email takes under 5 milliseconds, making the model suitable for inline MTA (Mail Transfer Agent) email-gateway filtering.
3. **Probabilistic outputs:** Logistic Regression produces posterior probabilities, $P(y = 1 \mid x) = 1 / (1 + e^{-w^T x})$, allowing administrators to set custom decision thresholds.
4. **Optimization:** The model was trained using the Stochastic Average Gradient Accelerated (`saga`) solver with an $L_2$ regularization penalty ($C = 1.0$), converging in just **4.9 seconds** across 131,094 samples.

## 6. Results & Evaluation

### 6.1 Test Set Classification Performance

The model was evaluated against the 32,774 unseen test instances. Performance was assessed via Accuracy, Precision, Recall, F1-Score, and macro/weighted averages (Table 6.1).

**Table 6.1:** *Test-set performance of the Logistic Regression classifier.*

| Evaluation Metric | Definition | Empirical Score |
|---|---|---|
| **Accuracy** | (TP + TN) / (TP + TN + FP + FN) | **98.60%** |
| **Precision (Phishing)** | TP / (TP + FP) | **98.34%** |
| **Recall (Phishing)** | TP / (TP + FN) | **98.97%** |
| **F1-Score (Phishing)** | 2 × (Precision × Recall) / (Precision + Recall) | **0.9865** |
| **Macro-average F1** | Mean F1 across both classes | **0.9859** |
| **Weighted-average F1** | Support-weighted F1 across both classes | **0.9860** |

## 6.2 Confusion Matrix Analysis

The confusion matrix (Figure 6.1) highlights the operational efficacy of the trained classifier.

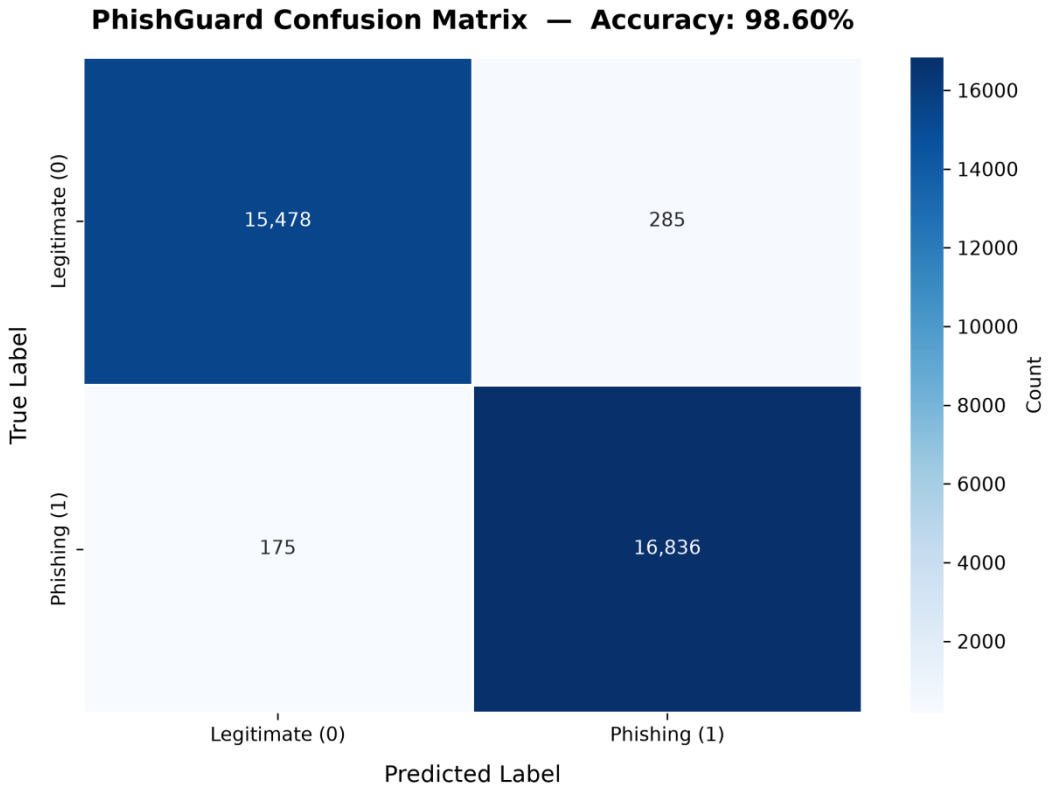



**Figure 6.1:** *Confusion matrix on 32,774 unseen test emails.*

**Quantitative breakdown:**

- **True negatives (TN):** 15,478 legitimate emails correctly identified as safe.
- **False positives (FP):** 285 legitimate emails flagged as phishing (**false-positive rate: 1.81%**).
- **False negatives (FN):** 175 phishing emails misclassified as benign (**false-negative rate: 1.03%**).
- **True positives (TP):** 16,836 phishing emails successfully detected (**detection rate / recall: 98.97%**).

In cybersecurity operations, a high recall (98.97%) means that nearly 99 of every 100 malicious emails are intercepted before reaching the user's inbox, while a low false-positive rate (1.81%) limits disruption to everyday legitimate business communications.

## 6.3 Qualitative Stress Testing & Concept Drift Analysis

To evaluate generalization beyond the static test split, the deployed model was subjected to qualitative adversarial stress testing across diverse contemporary email archetypes:

- **False-positive resilience (contextual disambiguation):** When tested against benign security notifications (e.g., an automated confirmation of a user-initiated password change containing high-risk tokens such as `password`, `security`, and `account`), the model correctly classified the message as **Legitimate**. Rather than over-indexing on isolated trigger words, the bigram features and the absence of coercive hyperlink structures prevented a false alarm.
- **Temporal concept drift & out-of-vocabulary (OOV) limitations:** While the classifier passed traditional spear-phishing, MFA reset, tax rebate, and SaaS billing fraud tests, it misclassified a modern **Web3 / DeFi smart-contract wallet-drainer** phishing prompt as benign. Because the

underlying benchmark corpora (Enron, CEAS 2008, SpamAssassin, Nazario) predate decentralized finance, domain-specific threat terms (`web3`, `liquidity pool`, `smart contract`, `defi`, `wallet drainer`) are absent from the fixed 10,000-token vocabulary. When the vectorizer drops these OOV terms, the remaining technical vocabulary (`vulnerability`, `patch`, `protocol`, `migrate`) statistically resembles legitimate IT security advisories in the training set.

- **Source-specific vocabulary:** As noted in Section 4.3, several of the most frequent tokens (e.g., `enron`, `cnn`, `ect`) are artifacts of individual source corpora. High test accuracy on a pooled split therefore partly reflects source-specific cues, and performance on emails from sources outside these seven datasets may be lower.

# 7. Challenges & Engineering Solutions

**Table 7.1:** *Technical challenges and the engineering solutions applied.*

| Technical Challenge | Root Cause | Engineering Solution Implemented |
|---|---|---|
| **Inconsistent CSV schemas** | Raw files used divergent headers (`body`, `text_combined`, `label`, `class`, `spam`). | Developed `find_column()` with priority exact-match lists and negative heuristic guards to reliably resolve content and target columns across all 7 sources. |
| **Encoding corruption** | Diverse SMTP mail transfer formats introduced mixed byte sequences (UTF-8, Latin-1, Windows-1252). | Implemented tiered encoding handlers with fallback replacement strategies (`encoding_errors='replace'`). |
| **Extreme outliers & skew** | Raw emails spanned from single-character fragments to multi-megabyte encoded log dumps. | Applied text sanitization, whitespace filtering, and logarithmic scaling during exploratory visualization. |
| **Resource & storage limits** | The host environment had near-capacity storage on the system partition. | Implemented automatic pip cache purges, isolated all virtual-environment dependencies to project-local paths on `D:\PhishGuard`, and configured `.gitignore` to prevent raw-data bloat. |
| **Data leakage prevention** | Vectorizing before the train-test split artificially inflates vocabulary overlap. | Structured `preprocess.py` to split text before fitting the `TfidfVectorizer`, preserving test-set isolation. |
| **Temporal concept drift (OOV)** | Legacy benchmark datasets lack emerging zero-day threat vocabularies (e.g., Web3/crypto scams). | Documented OOV token-dropping behavior during adversarial stress testing; outlined continuous retraining and subword transformer embeddings in the production roadmap. |

# 8. Interactive Deployment (Streamlit Application)

To transition PhishGuard from a research artifact to an interactive tool, an operational web application was developed in `src/app.py` using Streamlit.

## 8.1 Interface Features

1. **Dynamic artifact ingestion:** Uses `@st.cache_resource` to load `models/logistic_regression_model.joblib` and `models/tfidf_vectorizer.joblib` once at startup, keeping memory consumption low.
2. **Real-time text classification:** Users can paste raw email headers or bodies into a multi-line input area.
3. **Rotating multi-template benchmark suite:** Stateful cycling buttons (Template 1/N) rotate through diverse phishing scenarios (credential alerts, CEO wire fraud, courier redelivery scams,

HR payroll fraud, OneDrive harvesting) and benign workplace communications (team stand-ups, IT maintenance windows, all-hands memos, order confirmations) for live evaluation.

4. **Confidence breakdown:** Visual progress indicators display the percentage confidence for both the benign and threat hypotheses.

### 8.2 Local Launch Instructions

The interface can be launched locally from the terminal:

```
.\.venv\Scripts\streamlit run src/app.py
```

Upon execution, Streamlit starts the local web server at `http://localhost:8501`.

## 9. Conclusion & Future Roadmap

The PhishGuard project demonstrates that combining heterogeneous multi-source email datasets with standard NLP preprocessing and a regularized linear classifier produces a fast, effective threat classification engine. By curating 163,868 records across seven benchmarks, PhishGuard reduces the domain-specific bias of single-source training and achieves strong classification performance (**98.60% accuracy**, **98.97% recall**, **98.34% precision**) with a training time under five seconds. Stress testing also exposed its main limitations: out-of-vocabulary threat terminology from newer attack types, and reliance on source-specific vocabulary.

## 10. References


1. Klimt, B., & Yang, Y. (2004). *The Enron Corpus: A New Dataset for Email Classification Research*. In Machine Learning: ECML 2004, Lecture Notes in Computer Science, vol. 3201, pp. 217–226. Springer.
2. Androutsopoulos, I., Koutsias, J., Chandrinos, K. V., Paliouras, G., & Spyropoulos, C. D. (2000). *An Evaluation of Naive Bayesian Anti-Spam Filtering*. Proceedings of the Workshop on Machine Learning in the New Information Age, 11th European Conference on Machine Learning (ECML 2000), pp. 9–17. (Ling-Spam corpus.)
3. Nazario, J. *Phishing Corpus and Nigerian Fraud Corpus*. Publicly available email collections. https://monkey.org/~jose/phishing/
4. Apache SpamAssassin Project. *SpamAssassin Public Mail Corpus*. https://spamassassin.apache.org/old/publiccorpus/
5. Conference on Email and Anti-Spam (CEAS). (2008). *CEAS 2008 Live Spam Challenge* dataset.
6. Kaggle. *Phishing Email Dataset* (aggregated phishing and legitimate email collection). https://www.kaggle.com
7. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
8. Defazio, A., Bach, F., & Lacoste-Julien, S. (2014). *SAGA: A Fast Incremental Gradient Method With Support for Non-Strongly Convex Composite Objectives*. Advances in Neural Information Processing Systems 27 (NIPS 2014).
9. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
10. Streamlit Inc. *Streamlit Documentation*. https://docs.streamlit.io